

Adaptive HACT: Service-Matched Verification Evidence and Deployment Boundaries

This artifact candidacy draft accompanies the `future-code` research repository. Author names and affiliations require c

September 23, 2026 | Version 5.1 (evidence-serialization revision)

Abstract

Adaptive HACT v5.1 serializes verification evidence under fixed test execution schedules by separating immutable check identities from the mutable execution state that records when each check ran; identities and acceptance predicates are likewise held apart from the layout used to aggregate their completion records. Full completion hyperedges, rather than pairwise similarity alone, score feasible layouts; generation fencing preserves conditional publication soundness across permutations. We retain exact alphabetic dynamic programming as a reference, but make a learned-order compact balanced eight-ary tree the low-compilation-cost baseline and relegate the slower, empirically inferior block compiler to an ablation. A fresh, frozen-seed study contains 27 train/validation/held-out workloads. On clustered completions, the practical catalogue reduces held-out padded cost by 23.43%, 10.75%, and 5.56% at 128, 512, and 1,024 checks; independent and global controls give essentially no improvement. Re-aggregating twelve archived source candidates under three layouts produces 36 layout replays, not new code-generation or checker trials. The practical balanced layout reduces whole-epoch compressed bytes by 4.36–41.84% versus original-order fixed8, but remains larger than learned-order exact layouts. We derive an explicit, codec- and service-matched deployment test rather than claiming that historical export savings cancel a different harness’s verification overhead. Root-status inputs remain identical across layouts. BPE and physical-WAN measurements remain unavailable; we supply actual-tokenizer admission and authenticated TLS measurement tools without presenting local protocol tests as those missing experiments. The result identifies both a lower-cost implementation and a narrow, auditable operating region, not a universal agent or verification accelerator.

Keywords: bounded evidence; verification contracts; bandwidth-limited logging; completion hyperedges; cost-aware layout selection.

1 Productivity Boundary and Research Scope

This manuscript and its companion review were produced together. The governing research objective is to reduce the cost and time required for agents to complete large software projects correctly, without increasing coordination failures or weakening acceptance criteria. It is therefore useful to state, before the method, what the present evidence does—and does not—establish about that objective.

The certificate problem is not yet a productivity result. Adaptive HACT is an evidence-serialization component: it optimizes the *layout* that serializes verification evidence under fixed test execution schedules, separating immutable check identities from the mutable execution state that records a run. It does not change the agent’s planning, repository exploration, reasoning history, number of failed patches, or allocation of work; it does not schedule, reorder, or parallelize checks. A strong result on the certificate objective can therefore coexist with no improvement in whole-project productivity, and the manuscript acknowledges exactly this boundary throughout. Historical checker timings were essentially flat (39.69 s versus 39.77 s for fixed and learned layouts),

and a separate small harness paid roughly five seconds per episode for strict verification versus direct pytest. These are *negative controls*: they show that a smaller certificate transcript is not automatically a faster build, a better agent, or a smaller root-status prompt, and they delimit the serialization claim rather than support any acceleration claim.

Where the measurement stops. No hosted model was called; no physical WAN and no actual public-vocabulary BPE encoding were evaluated; and there is no new SWE-bench or natural historical-bug cohort. The 288 loopback transfers belong to an earlier version and use application pacing rather than real network conditions. Consequently this paper does not claim a latency, token, or billing win. Its contribution is a scoped systems component: when protected hierarchical evidence must cross a constrained link, a learned-order balanced layout can reduce the bytes that must cross it, and we provide the accounting, the codec-matched test, and the honesty about what remains unmeasured.

The direction implied by the review. The review recommends treating this certificate result as the evidence *backend* of a larger program about verified-frontier context retirement, dependency-aware parallelism, and task granularity. That program is a design and evaluation protocol in progress, not a completed experiment; Section 11 of the review and our discussion articulate the concrete experiment that would promote this component to a productivity claim. We do not relabel the scripted evidence below as an evaluation of that larger controller.

2 Introduction

Many-agent systems need a reliable answer to “which declared checks hold on this exact candidate?” Their task graph and conversational organization need not be the layout that aggregates this evidence. HACT separates the two: immutable check identities determine the acceptance contract, while a bounded-packet tree represents intermediate results. A smaller tree transcript is not automatically a faster build, a better agent, or a smaller root-status prompt.

The distinction matters economically. Historical source timings were 39.69 versus 39.77 seconds for fixed and learned layouts. A separate small harness paid roughly five seconds per episode for strict verification compared with direct pytest. Meanwhile, a historical compressed loopback export saved approximately 2.21 seconds across twelve source streams. These different cohorts cannot be combined into a measured end-to-end speedup, and neither observation justifies adding a strict wrapper solely to save transmission time.

Three deployment decisions, not one. A trusted root-status consumer needs neither a learned tree nor all intermediate packets. A deployment that already requires protected verification and hierarchical evidence can compare fixed and learned layouts at marginal cost under that unchanged service. Adopting protected verification where ordinary pytest was sufficient is a different decision: its additional preparation and checking costs must be justified independently. We make this service distinction explicit in both the paper and a fail-closed planning API.

The simpler compiler is now primary. The prior block compiler was 1.54–5.34% more expensive in held-out bytes than a compact balanced eight-ary tree on the same learned order. We therefore remove block DP from the practical path. The new default catalogue retains the incumbent and proposes a balanced tree on a training-learned average-linkage order; separate validation compares full-wave costs. Exact DP remains a useful conditional optimum and reference. It can still produce smaller evidence than the practical default, as our source replays show. Low fitting cost and lowest byte cost are different objectives.

Core contributions and architecture. Adaptive HACT v5.1 serializes verification evidence under fixed test execution schedules. Its architecture separates two kinds of state that many systems conflate. *Immutable check identities*—registry IDs, acceptance predicates, and the candidate binding—define what must hold for a candidate to be accepted; they are fixed when the registry and candidate identity are published and are never rewritten by execution, layout choice, or learning. *Mutable execution state*—tokens, timestamps, generation counters, and the order in which checks actually ran—records one run of those checks and may differ run to run. A result token carries the immutable semantic ID plus the run’s binding; a layout-specific kernel maps stable IDs to display positions. Because identities are stable, permuting the layout or replaying under a different schedule moves records without changing their meaning (Section 7). The execution schedule σ_e is an input the serializer must respect, not a variable it optimizes: HACT changes how evidence from an unchanged schedule is laid out and aggregated, never when checks run or what their outcomes mean. The substantive research contribution remains hyperedge-scored, identity-preserving verification aggregation and its conditional guarantees. This revision resolves an implementation choice, adds service-matched cost accounting, and evaluates the no-DP practical catalogue on fresh frozen synthetic splits. It separately replays archived source evidence and provides tools for the unmeasured token and WAN boundaries. Clustering, balanced trees, alphabetic DP, compression, break-even arithmetic and TLS are established techniques, not new inventions claimed here. The artifact is a tested research implementation; its tests do not establish production readiness or top-conference acceptance.

Positioning: evidence serialization, not agent acceleration. Every design decision in v5.1 follows from a deliberately narrow scope: HACT is an evidence-serialization component. It reduces the bytes needed to publish which declared checks hold, under a delivery service that already requires protected verification and hierarchical evidence. It does not schedule checks, does not parallelize work, does not alter the agent’s planning or repository exploration, and does not claim that a smaller transcript makes any agent faster or more productive. Where the review’s larger program—verified-frontier context retirement, dependency-aware parallelism, task granularity—would use this certificate as a backend, that program is an evaluation protocol in progress (Section 1); its experiments have not been run. The negative controls in Section 1 exist precisely to keep this boundary honest: flat checker timings and per-episode verification overhead are reported as limits of the serialization claim, not evidence against it or support for an acceleration claim.

3 Related Work and Positioning

Testing, impact analysis, and incremental computation. Test prioritization seeks earlier fault exposure [1]; regression-test selection uses change dependencies [2, 3]. The earlier IC-HACT ranker uses controlled mutation outcomes to prioritize checks, an instance of established fault-exposure methods. It is held fixed in the new source comparisons. Self-adjusting computation and build-system research already formalize dependency-aware recomputation [4, 5]. In contrast to test selection, our strict pass requires the entire approved registry on the identified candidate. We do not reproduce NameRTS or Ekstazi and claim no superiority over them.

Seriation and authenticated layouts. Spectral seriation orders items using an affinity matrix [8]. Average-linkage clustering likewise provides an established proximity-based ordering. Generalized alphabetic-tree optimization supplies the subtree/forest recurrence used here [6]. Workload-adaptive authenticated structures, including Huffman–Merkle Trees [7], already optimize layouts under access patterns. Our price counts repeated completion-frontier updates, including strict-epoch construction, under a fixed packet cap. Pairwise affinity proposes an order; full

hyperedge observations score it. No global optimality or unproved NP-hardness result is claimed for unrestricted joint ordering and tree synthesis.

Online decisions with switching costs. Fixed-share learning tracks changing experts [9]; dependent sampling reduces switches compared with independent weighted-majority draws [10]. Full-information and bandit switching-cost settings have different achievable regret guarantees [11]. We use the full-information setting because one recorded completion wave can be scored against every frozen certificate layout without reexecuting the tests. This is not full counterfactual feedback for arbitrary agent strategies: if a layout changes the tests’ execution schedule or model behavior, our exogenous-cost assumption may fail. Our bound specializes established tools to measured layout-installation costs; it is not a new minimax online-learning result.

Agent runtime assurance and evaluation. Proof-Carrying Agent Actions [12] and verification-portfolio commit gates [13] address related action-governance problems. Our local certificates are neither signatures nor zero-knowledge proofs. SWE-bench evaluates real repository issues and patches [14]; our source mutations and scripted I/O repairs must not be called SWE-bench results. The current integration measures real orchestration and verification, not model reasoning quality.

Logging and serialization controls. Telemetry systems already batch and compress records [15]. HACT is not a replacement for those techniques: fewer padded bytes need not imply fewer compressed bytes. We measure both. Moreover, flat result ledgers and a trusted root-status endpoint can be much smaller than an archive of all intermediate aggregation updates; they satisfy different delivery needs and must not be hidden from the comparison. Several closest lines of agent work—observation masking [16], context folding [17], and decentralized shared context [18]—motivate the productivity boundary in Section 1 rather than being reproduced here.

4 Model and Trust Boundary

4.1 Semantic registry, execution, and display

Let $\mathcal{R} = (g_0, \dots, g_{n-1})$ be a nonempty approved registry of unique check IDs. The candidate identity is

$$e = (\text{epoch}, \text{snapshot}, \text{checker}, \text{environment}, \text{registry}). \quad (1)$$

The snapshot covers source and protected acceptance inputs. The checker and environment are explicitly bound; an epoch increases even after an ABA return to identical source bytes. Only a trusted runner submits final PASS, FAIL, or UNKNOWN records. Missing, stale, skipped, incompatible, or timed-out outcomes are not passes.

Immutable identities versus mutable execution state. The model splits its state into two disjoint layers. *Immutable check identities* are the registry $\mathcal{R}$, the candidate identity e , and the acceptance predicate each check declares; they are fixed at publication and are never rewritten by execution, layout choice, or learning. *Mutable execution state* is everything a run may change: the result tokens, their generation counters, epoch-local timestamps, and the execution permutation σ_e that determines when checks run. A record’s identity is its semantic ID bound to e ; its display position is not part of that identity. This separation is what makes evidence serialization under fixed schedules well-posed: the schedule and outcomes are observed inputs, and only the aggregation layout is chosen. No layout or learner writes execution state, and no execution mutates the registry or acceptance predicates.

An execution permutation σ_e determines when checks run. A layout $L = (\pi, T)$ independently maps tree positions to semantic registry indices. For a node covering positional interval $[i, j]$, its semantic coverage is

$$G_L(i, j) = \{g_{\pi(i)}, \dots, g_{\pi(j)}\}. \quad (2)$$

The stored layout must contain an exact permutation, and every internal node’s children must partition its positional interval without gaps or overlaps. The task graph is outside this mathematical tree; one worker can discharge many checks, and one integration check can cover several workers.

The acceptance predicate is independent of both permutations:

$$A(e) = \mathbf{1}\{\text{every } g_i \in \mathcal{R} \text{ has trusted, matching, current PASS evidence for } e\}. \quad (3)$$

An observed fail permits rejection before all checks finish. It does not assert coverage of the unexecuted remainder. Acceptance is relative to the approved predicates, not proof of all program behavior.

4.2 Packet and lifecycle costs

Each internal node of arity d emits a fixed-width packet

$$c(d) = H + Sd, \quad 2 \leq d \leq b = \lfloor (C - H)/S \rfloor. \quad (4)$$

Here $H = 512$, $S = 320$, and $C = 3072$ bytes, so $b = 8$. A mandatory field that exceeds its slot is rejected, not silently truncated. The cap applies to one certificate packet, not to all project memory, an entire registry manifest, or every LLM prompt. A manifest is a referenced control-plane artifact; its measured size is charged on migration.

Let $W_{e,1}, \dots, W_{e,q_e}$ be the semantic IDs whose final records become available between successive refreshes. For a strict new candidate epoch, the observed cost is

$$\ell_e(L) = W(L) + \sum_{a=1}^{q_e} \sum_{v \in \text{int}(T)} c(d_v) \mathbf{1}\{W_{e,a} \cap G_L(v) \neq \emptyset\}, \quad W(L) = \sum_v c(d_v). \quad (5)$$

The first term constructs the full initially unknown tree. The CLI batches an executed prefix into groups of eight, flushing its final partial group. A successful strict epoch runs the whole registry. This conservative implementation does not reuse a pass across distinct source snapshots.

We separately study persistent-state refresh traces that omit $W(L)$ at each tick, as in a cache with correctly specified incremental dependencies. Each topology installation is still cold and explicitly priced. These traces evaluate the controller’s cost model; they are not the strict candidate-gate timing experiment. They do not demonstrate safe dependency inference from arbitrary code.

4.3 Why more than invalidation or pairwise similarity is needed

A classical result shows that marginal check activation cannot identify the best aggregation tree under joint invalidation (Appendix E). A further result shows that even the full initial invalidation distribution cannot identify optimal progressive verification: early rejection and completion batching change the update sequence. Both distinctions remain relevant after allowing permutations.

Proposition 1 (Pairwise affinity is insufficient for full-wave scoring). *There exist two distributions with identical singleton and pairwise co-activity probabilities, hence identical Jaccard affinities, but different activation probabilities for the same three-check subtree.*

Proof. On IDs $\{0, 1, 2\}$, let P be uniform on $\emptyset, \{0, 1\}, \{0, 2\}, \{1, 2\}$ and Q uniform on $\{0\}, \{1\}, \{2\}, \{0, 1, 2\}$. Every singleton has probability $1/2$, every pair has joint probability $1/4$, and its Jaccard affinity is $1/3$. Yet the probability that the triple is hit is $3/4$ under P and 1 under Q . Thus a packet emitted on that interval has different expected cost. $\square$ $\square$

This elementary parity construction is not a new fact about higher-order statistics. Its implication for the compiler is specific: a pairwise clustering objective cannot substitute for scoring recorded completion hyperedges. It does not, by itself, prove that these two distributions have different globally optimal layouts.

5 Learning Layouts Without Changing the Contract

5.1 Practical catalogue and full-wave selection

Training waves define counts N_i , N_{ij} and affinity

$$a_{ij} = \frac{N_{ij}}{N_i + N_j - N_{ij}}, \quad (6)$$

with zero for an unobserved union and zero diagonal. The practical proposal uses established average-linkage clustering: join the closest current clusters, choose endpoint orientation by cross-boundary affinity, and break ties canonically. All semantic IDs, including disconnected and unseen checks, remain included exactly once.

Given the learned order, group consecutive leaves into blocks of at most eight, promote a singleton unchanged, and repeat upward. This bottom-up compact balanced construction does not call a tree optimizer. The catalogue retains the incumbent first, includes original-order fixed8 when distinct, and proposes learned-order fixed8. Identical layouts are deduplicated. A separate validation cohort scores complete completion waves, including initial construction; ties retain the incumbent. The API returns an advisory choice, not deployment authorization. The registry and packet cap cannot change during selection.

For any proposal π , training episodes supply interval weights

$$r_\pi(i, j) = 1 + \frac{1}{m} \sum_{e=1}^m \sum_a \mathbf{1}\{W_{e,a} \cap G_L(i, j) \neq \emptyset\}. \quad (7)$$

These weights also define the exact-DP reference below. Pairwise affinity proposes an order; it does not replace the full-wave cost. The practical catalogue contains at most three candidates and uses no DP. Tree construction is $O(n)$ after ordering, not a linear-time claim for affinity construction, clustering or validation scoring. The reference implementation retains spectral ordering and exact fitting for research comparisons; neither is invoked by the practical default.

5.2 Exact conditional tree synthesis

Let $D(i, j)$ be the optimal subtree cost and $F_k(i, j)$ the optimal cost of an ordered forest with k nonempty children. With infeasible states equal to $+\infty$,

$$D(i, i) = 0, \quad (8)$$

$$F_1(i, j) = D(i, j), \quad (9)$$

$$F_k(i, j) = \min_{i+k-2 \leq u < j} \{F_{k-1}(i, u) + D(u+1, j)\}, \quad (10)$$

$$D(i, j) = \min_{2 \leq k \leq \min(b, j-i+1)} \{r_\pi(i, j)c(k) + F_k(i, j)\}. \quad (11)$$

Theorem 2 (Exactness conditional on order). *For a fixed π , nonnegative interval weights, and the packet model in Equation (4), Equation (11) returns a minimizing feasible tree in $O(bn^3)$ time and $O(bn^2)$ space.*

Proof. Every feasible root has an arity and a contiguous partition of its interval. Its local contribution depends only on that interval and arity; all remaining contributions belong to independent child subtrees. The forest recurrence enumerates every such partition. Induction on interval length proves exactness. There are $O(bn^2)$ states, each scanning at most n splits. The apparent $F_1 = D$ dependency is used only on shorter intervals when constructing a forest with at least two components. $\square$ $\square$

This is a specialization of classical alphabetic-tree optimization [6]. The historical three-proposal reference multiplies compilation time by a constant, not by $n!$. Dense spectral computation can itself cost $O(n^3)$; the reference clustering implementation uses $O(n^2 \log n)$ heap operations and $O(n^2)$ storage.

Proposition 3 (Validation after learned ordering). *Condition on a training set that fixes K complete layouts. For m independent identically distributed validation epochs with each layout cost in $[0, B]$, let $\hat{L}$ minimize empirical validation cost. With probability at least $1 - \delta$,*

$$J(\hat{L}) \leq \min_{L \in \mathcal{C}} J(L) + 2B \sqrt{\frac{\log(2K/\delta)}{2m}}. \quad (12)$$

Proof. Conditionally, the catalogue is fixed. Hoeffding’s inequality and a union bound over K layouts bound every empirical mean error by the displayed radius without its factor two. Inserting empirical costs on either side of the empirical minimizer yields the result. $\square$ $\square$

The guarantee is only relative to the fixed catalogue. It neither justifies choosing among learned permutations on their training errors alone nor covers distribution shift. We do not report this potentially loose bound as an empirical confidence interval; reported intervals use independent experimental seeds.

5.3 Compiler identity and scope

The practical default has no hidden microtree optimization. Exact DP is a reference and optional offline candidate. The block-backbone proposition and implementation remain in Appendix D solely as an ablation; its restricted-class exactness does not override the observed simpler-baseline advantage. At 1,024 checks, order learning still takes seconds in our measurement, so layouts should be cached and compiled away from the agent’s per-action path. Validation of padded costs does not optimize compressed archives or model tokenization; adoption must measure the actual delivery representation.

6 Practical Online Selection: Price the Window, Not the Promise

Freeze a compiled catalogue $\mathcal{C} = \{L_1, \dots, L_K\}$. At event t , choose a layout before observing the current completion wave; afterwards score that same wave under every catalogue member. This is full-information feedback only because the check schedule is held fixed. It is not counterfactual feedback for arbitrary agent behavior.

A different layout is installed by serializing its canonical manifest and reconstructing its complete certificate tree:

$$M_{ij} = \begin{cases} 0, & L_i = L_j, \\ |\text{manifest}(L_j)| + W(L_j), & L_i \neq L_j. \end{cases} \quad L_T = \sum_t \ell_t(A_t) + \sum_{t=2}^T M_{A_{t-1}, A_t}. \quad (13)$$

Algorithm 1 Deployable evidence control; inference and checking remain separate**Require:** Protected registry, checker and snapshot identities; frozen layout catalogue

```

1: for each event do
2:   Choose from past evidence using Equation (14), or keep the frozen layout
3:   if layout changes then
4:     Revoke tickets, increment generation, remap stable IDs, rebuild and price the installation
5:   end if
6:   Receive trusted completion records for the fixed check schedule
7:   Reject stale, conflicting or malformed records; aggregate bounded packets
8:   Publish only the complete current contract result; otherwise retain FAIL/UNKNOWN
9:   Expose a root-status card; retrieve diagnostic packets only when needed
10:  Score the observed event and append its cost vector to the bounded window
11: end for

```

The initial installation is separate. These byte prices match the evaluated uncompressed migration protocol. A deployment using compression must measure its own priced wire representation; the DP objective is not automatically an optimum for a compressed log.

The primary practical rule. Let $\bar{\ell}_{t,i}$ be the mean cost on the last $\min(w, t - 1)$ observed events, and $j_t = \arg \min_i \bar{\ell}_{t,i}$ with deterministic ties. Starting from the calibrated incumbent a , use the frozen rule

$$A_t = \begin{cases} j_t, & w(\bar{\ell}_{t,a} - \bar{\ell}_{t,j_t}) > M_{a,j_t}, \\ a, & \text{otherwise,} \end{cases} \quad w = 64. \quad (14)$$

With no history, keep a . A tie in estimated savings does not trigger migration. During warm-up, the forecast horizon remains w even if fewer events were observed, matching the original baseline. This is an empirical break-even forecast, not proof that past savings repay a future switch. It can lose under abrupt reversal; preserving the frozen layout is always a legitimate alternative.

The runtime API enforces `choose()` before `observe()`, bounds history to 64 vectors, rejects nonfinite costs, and checkpoints only at event boundaries. Replaying all 80 archived streams, with periodic checkpoint/resume, must reproduce the original baseline’s full action and byte-cost paths exactly. The exercise validates implementation equivalence, not a new performance dataset.

Why fixed-share is no longer primary. On the archived stationary, long-shift and rapid-shift streams, the coupled fixed-share policy is more expensive than the priced window. Its standard tracking analysis therefore moves to Appendix C, with all negative results retained. We do not manufacture an adversarial workload merely to make that policy win. Neither the appendix bound nor the empirical window result establishes general superiority over other switching-cost policies.

7 Stable Evidence Through Permutation and Migration

A result token carries (epoch, g_i, b_i) , where b_i binds the semantic ID to the registry, source snapshot, checker, and environment. It does not carry a display position as its identity. A layout-specific kernel maps stable IDs to current positions. During migration it rebuilds from canonical trusted records; it never transplants an ordinal cache entry into a different permutation.

Publication tickets additionally carry a monotone layout generation and layout hash. A layout switch revokes the issued ticket, advances the generation, and forces reconstruction. Returning to an earlier tree therefore cannot revive that tree’s old ticket. An in-flight token may remain valid through a layout-only change, but a new candidate epoch fences it. Duplicate identical evidence is idempotent. Conflicting final records in one epoch revoke authorization: a recorded fail remains

a fail; otherwise the obligation becomes unknown. Resolution requires a new epoch, not a more persuasive agent summary.

Theorem 4 (Layout-independent local publication soundness). *Assume complete approved predicates and dependency identities, a trusted checker executing immutable identified inputs, collision-resistant binding, and atomic local epoch/generation checks. For any sequence of valid layout permutations, any advisory learning outputs, and any delivered evidence order, authorization implies Equation (3). If each checker is sound for its declared predicate, those predicates hold on the authorized candidate.*

Proof. A canonical pass requires a current token for the same semantic ID and immutable identities. Permutation is a bijection, so rebuilding moves each such record to exactly one position without changing its meaning. Exact child partitioning preserves coverage and pass/fail/unknown counts by induction up the tree. The root can be authorized only with $(n, 0, 0)$ and matching current registry, epoch, generation, and layout. Old tickets fail those checks. Conflicts prevent authorization. No choice made by the learner writes trusted outcomes or changes the registry. $\square$ $\square$

This is a conditional local guarantee, not remote attestation or distributed consensus. An incomplete suite, compromised checker, omitted dependency, or nondeterministic external service can invalidate a broader correctness claim. The CLI's file copy is not an OS sandbox. An external merge/deployment service must atomically compare the verified candidate and integration-head identities at commit; a report file alone is not that transaction.

8 Implementation and Cost Accounting

The artifact adds `practical.py`, `deployment.py`, `token_budget.py` and `tls_probe.py`. The practical CLI produces a balanced catalogue without calling either exact or block DP. The historical exact and block compilers remain separately callable. The existing stable-ID guard, priced window, codec and Future Code bridge are retained. The strict CLI compares protected input sets and hashes, detecting added tests or configuration files as well as changed and deleted ones. Schema-2 contracts bind the approved checker and environment and refuse silent reuse of legacy contracts. Source manifests are checked before and after snapshotting and verification; mutation of an observed snapshot prevents a pass.

The bridge runs as an ordinary subprocess acceptance gate in the unmodified Future Code v1.2 runtime. The supervisor still owns worker leases, private attempts, bounded contexts, and serialized integration. Non-pass exits cause candidate rejection. Detailed evidence and binary certificate packets live outside candidate write scopes; only a bounded JSON result card returns to the worker. The plug-in accepts a precompiled layout. The online selector and migration guard are separately integrated and stress-tested as a library path; a live, self-retuning hosted deployment was not evaluated.

A different scalar objective could combine these quantities only with declared unit conversions and observable counterfactual costs. This implementation does not optimize unknown model latency or billing. The exact-byte claim also depends on the stated padded format; compression defines a different, potentially nonseparable objective for which this DP is not claimed exact.

9 Service-Matched Delivery and Admission

9.1 Do not conflate three consumers

The local aggregator receives bounded packets; a logging collector may receive their complete stream; a supervisor normally receives a status projection and retrieves details selectively. Define

Table 1: Separate ledgers avoid converting byte savings into unsupported latency or token claims.

Measurement	How obtained	In optimized objective?
Certificate bytes	Actual padded packets; independently reconstructed from waves	Yes
Migration bytes	Canonical layout manifest plus forced full rebuild	Yes, online traces
Fit and scoring time	Monotonic-clock measurements	Reported separately
Preparation, copying, checking, aggregation	Wrapper timers and outer call duration	Reported separately
Active harness episode	Supervisor startup through completion and shutdown	Reported separately
Prompt bytes and construction time	Captured actual HTTP messages and context builder	Reported; not tokens
Hosted token usage and API billing	No provider inference was run	Unknown, never zero-imputed

$\psi(e, c)$ as canonical status text containing semantic identity, counts and local authorization, but no display positions. For the same trusted semantic report,

$$\psi_{L_1}(e, c) = \psi_{L_2}(e, c). \quad (15)$$

Its byte length and token count under any fixed deterministic tokenizer are therefore identical. This identity is not a new theorem about tokenization. It supplies a negative control: layout savings cannot reduce a layout-independent root input. The card is a projection of a trusted report, not authority to accept arbitrary worker JSON.

Diagnostics preserve full header/card records and page whole packets under a 16,384-byte limit. A packet that cannot fit is rejected, not truncated. For a real supplied tokenizer τ and explicitly reserved framing budget r , the new admission API additionally requires

$$|p|_{\text{UTF8}} \leq C_{\text{bytes}}, \quad |\tau(\text{decodeUTF8}(p))| + r \leq C_{\text{tokens}}. \quad (16)$$

The implementation retokenizes each full proposed page, including its prefix. Token counts are not assumed additive across concatenations. Greedy packing guarantees the tested limits, not a minimum number of pages. Toy counters in API tests are not BPE measurements. The optional measurement command requires actual tiktoken software and vocabulary [19], fingerprints the encoding, and records per-page content hashes. Missing encodings produce UNKNOWN and null counts; there is no byte-ratio fallback. Provider framing and billed usage require a separately measured adapter, not a guessed constant.

9.2 Encoding and the simplest sufficient service

Canonical unpadded JSON, independently compressed packets, and whole-epoch DEFLATE preserve their input records with different operational boundaries. Whole-epoch compression requires the completed archive, whereas per-packet compression permits incremental export. The padded objective is not an exact optimizer of either compressed bytes or tokens. Practical selection is advisory until its gain is measured with the deployed codec.

For a trusted consumer needing only final status, a root endpoint or flat ledger can be much cheaper than the intermediate stream. This changes the diagnostic service, not necessarily the underlying check set. Comparing that smaller service with an exhaustive stream cannot demonstrate that a tree optimizer is efficient or necessary. The strict checker, data representation, and delivery demand must each be named explicitly.

9.3 Adoption cost is not marginal layout cost

Consider q copies of the same workload and delivery service through one nonoverlapped bottleneck. Let B_0, B_1 be codec-matched application bytes per copy, $R > 0$ effective application goodput, $E \geq 0$ additional encoding/decoding work per copy, and $C \geq 0$ consistently amortized one-time incremental work. Common setup cancels only when actually shared. The conditional net benefit is

$$d = (B_0 - B_1)/R - E, \quad \Delta(q) = qd - C. \quad (17)$$

A strictly profitable integer copy count exists only when $d > 0$, and then

$$q_{\min} = \lfloor C/d \rfloor + 1. \quad (18)$$

This is accounting, not a new networking or regret theorem. A zero gain retains the baseline. The implementation uses decimal arithmetic at equality and rejects mismatched service IDs. Missing C , E or R returns UNKNOWN, not zero. An explicit zero is permissible only as a declared assumption or measurement.

When both variants already use the same protected gate, C is extra layout work, not the full strict-wrapper cost. When adopting strict verification from ordinary testing, its additional cost must be justified independently; the services differ and the planner returns INCOMPARABLE. Repeating exports amortizes one candidate’s preparation, not the preparation of future new candidates. Parallel links, computation overlap, cached diagnostics, queueing and different connection policies require direct critical-path measurement. High RTT alone does not make byte savings larger if both exports pay the same setup and request count.

9.4 Remote measurement and security boundaries

The historical transfer experiment is receiver-paced loopback, not a WAN. The new opt-in measurement kit uses verified TLS server certificates and hostname checking; upload and nonlocal bind require explicit switches. Frames and decompressed archives are bounded, and acknowledgments check received length and digest. Local tests exercise cold and reused TLS sessions, rejected hostnames, untrusted roots, malformed acknowledgments and truncated records.

These tests validate a protocol path only. They provide no physical-WAN throughput, jitter, packet-loss or retransmission measurement. The supplied protocol records host and path provenance, codec, connection policy and matched payloads, and requests separately observed RTT/goodput and transport diagnostics where available, consistent with the distinctions in TCP throughput testing [21]. Fields that the minimal client cannot observe remain null. There is no receiver-side artificial pacing in a qualifying WAN trial. No public upload, account creation or credential search was performed.

TLS authenticates the configured server, not the checker’s truthfulness. Protected reference data, immutable candidate identity and local monotone state remain required for Theorem 4. There is no remote attestation, Byzantine consensus, client authorization service, non-repudiation or OS sandbox. Replacing both evidence and its trusted reference still defeats a digest-only comparison. The receiver is a bounded measurement utility to use behind an operator-controlled firewall, not a production ingestion service.

10 Evaluation of the Practical Revision

10.1 Provenance and frozen questions

The v5 contract was saved before the new runs. It fixes three sizes (128, 512, 1,024), three completion families (clustered, independent, global), and three seeds (55011–55013). Each of the

Table 2: New v5 synthetic study. Reduction is selected-layout held-out padded cost relative to the original-order balanced incumbent; positive is better. Fit includes affinity/order learning and catalogue construction, not validation scoring. Three seeds per row; ranges are descriptive.

Checks	Family	Mean (%)	Range (%)	Fit median (s)	Changed
128	cluster	23.43	22.40–24.45	0.018	3/3
128	independent	0.00	0.00–0.00	0.016	0/3
128	global	0.00	0.00–0.00	0.032	0/3
512	cluster	10.75	10.06–11.13	0.523	3/3
512	independent	0.07	0.00–0.20	0.548	1/3
512	global	0.00	0.00–0.00	0.645	0/3
1,024	cluster	5.56	5.16–5.80	2.991	3/3
1,024	independent	0.00	0.00–0.00	3.086	1/3
1,024	global	0.00	0.00–0.00	3.462	0/3

27 workloads has 128 training epochs, 128 validation epochs and 256 held-out epochs. Random draws are sequential disjoint samples; deterministic global observations naturally coincide. The catalogue is fit only on training data, selected on validation, and reported on held-out data. Cold strict-epoch construction is included in every objective. The nine size/family summaries use seed means and observed ranges, not episode-pooled significance tests.

A separate study re-aggregates twelve archived source candidates under three layouts, producing 36 layout replays. All use the same original-order trusted check records, the same executed sequence and batches of eight. Average-linkage orders were learned on the earlier training cohort and frozen before these candidates. None of these replays is a fresh checker execution, independent code repair, or new agent trial. Historical v3/v4 data remain in the appendices with their original cohort identities.

10.2 Practical default without exact or block DP

On clustered completions, mean held-out reductions are 23.43%, 10.75% and 5.56% as registry size increases (Table 2). Global updates give no reduction; independent updates give zero except for one of the three 512-check seeds, whose small gain produces a 0.07% mean. These controls do not support universal value from ordering. The selected method includes no DP, and a regression test makes any call to the exact fitting helper fail on this path.

The full catalogue fit median at 1,024 checks remains about 3 seconds; order learning is not free. The fixed construction can reuse a previously approved order in linear work, but no linear-time claim is made for the full learned pipeline. The smaller percentage at larger registries is consistent with substantial layout-independent cold construction in this benchmark; it should not be interpreted as a learned scaling law.

An initial execution exceeded its 120-second tool limit after saving 26 complete records. No process remained; explicit resume skipped those exact records and completed the remaining frozen seed without changing data or selection. All 27 observations and the interruption record are retained.

10.3 Source-evidence replays: simplicity is not byte optimality

Learned balanced layouts reduce whole-epoch compressed bytes by 41.84% for NetworkX, 4.36% for Toolz and 18.53% for Future Code (Table 3). These are not the larger v4 exact-layout savings relabeled as results of the new default. The balanced representation remains larger than learned exact in every project here. Exact fitting may therefore be appropriate when amortized delivery justifies its additional compilation; the practical default trades that potential gain for simpler

Table 3: New serialization of twelve archived source candidates, four per project. The first three columns compare learned-order balanced8 with original-order fixed8. The last column compares learned balanced with learned exact after epoch compression: positive is a penalty. All include the codec’s application framing.

Project	Padded (%)	Canonical (%)	Epoch (%)	vs. exact penalty (%)
networkx	48.63	48.74	41.84	11.68
toolz	8.78	8.70	4.36	1.52
future	29.12	29.23	18.53	6.67

Table 4: Conditional minimum profitable copies of the same twelve-candidate cohort with whole-epoch compression. $E = 0$ is an assumption. The columns assume extra cost per candidate, paid once; cohort costs are 0.6, 6 and 60 seconds. These are not measured strict-wrapper overheads.

Assumed goodput	0.05 s/candidate	0.5 s/candidate	5 s/candidate
8 KiB/s	1	1	5
64 KiB/s	1	4	34
1 MiB/s	6	54	538

fitting.

Across the twelve cases, batch-compressed application bytes are 416,046 for fixed8, 299,081 for learned balanced and 278,831 for the re-serialized learned exact layout. Semantic counts, verdicts and local authorization agree across all 36 replays. Their provenance binds the archived check records by digest. Serialization includes newly bound layout/run metadata; numerical bytes need not exactly reproduce a prior archive’s compressibility. The independent audit checks actual emitted bytes rather than substituting earlier totals.

Root-status text is identical in all twelve triples and occupies 593–594 UTF-8 bytes. Exhaustive diagnostic pages total 148 for fixed8, 95 for learned balanced and 87 for learned exact. The new default therefore uses 35.81% fewer pages than fixed8, not v4’s 41.22% exact-layout reduction. The largest emitted page is 16,364 bytes under a 16,384-byte limit. Reading every page is a specified exhaustive diagnostic service, not observed LLM behavior. There is no measured token reduction; root token equality follows from identical text for a fixed tokenizer, but no numerical BPE counts are asserted.

10.4 Codec-matched deployment scenarios, not measured speedups

The measured fixed-to-balanced compressed byte difference is 116,965 per twelve-candidate cohort. Equation (17) gives 1.784744 seconds of gross delivery saving per cohort copy at an assumed 64 KiB/s goodput, with $E = 0$. This is arithmetic from replayed byte sizes, not the older 2.21-second measured loopback saving of a different layout.

The 5-second column (Table 4) is a deliberately declared sensitivity scenario, not an estimate transferred from the historical harness. At 64 KiB/s and one export per candidate, the assumed 60 seconds of cohort overhead overwhelms the approximately 1.785-second delivery saving. Repeated delivery might change the equation, but the application must actually require it. With new candidates on each repetition, preparation recurs and cannot be amortized as the same C . All 27 scenarios (three codecs, three rates, three overhead assumptions) are available, including negative outcomes. The planner rejects mismatched services and missing measurements rather than inventing a benefit.

10.5 Historical controls remain controls

The earlier 288 loopback transfers belong to v4, use application pacing rather than real WAN conditions, and are not rerun or relabeled as v5 trials. For twelve compressed fixed/exact exports at 64 KiB/s, their measured totals were 6.624 and 4.413 seconds. The larger uncompressed totals were 33.499 and 18.954 seconds. Compression of fixed8 was more impactful than selecting an uncompressed learned layout. None of those durations is an IP/TLS throughput measurement or predicts a loss-prone path.

The original 39.69/39.77-second checker totals and 12.98/18.11/18.12-second median direct/fixed/learned harness durations remain negative controls. They came from different source/harness fixtures, so this paper does not subtract their times across cohorts. The historical window-controller and block-compiler evidence is retained unchanged. Block DP’s 1.54–5.34% penalty to same-order balanced8 is the reason it is now an ablation, not a contribution advertised as deployment acceleration.

10.6 Unmeasured endpoints and tested interfaces

Both requested BPE encodings, `cl100k_base` and `o200k_base`, remain UNKNOWN: the software was absent, package/vocabulary fetches failed, and a connector-retrieved build artifact was expired. No substitute tokenizer, model alias or byte division was used. The measurement tool records exact content-only counts when a real encoding is available; its unit tests use explicitly labeled toy counters only to validate admission logic.

No two authorized remote measurement hosts or designated live model endpoint were available. Physical-WAN and hosted-LLM trial counts are zero. The new TLS utility’s local compatibility tests check transport integrity, certificate rejection and bounded framing; they do not measure WAN jitter/loss or close the agent-evaluation gap. A reproducible deployment protocol is supplied, rather than a claimed remote experiment.

10.7 Software and independent audit

The v5 prototype passes 329 tests, including 76 new cases; the existing version-equality assertion is updated to require 5.0.0 consistently in runtime and package metadata. A duplicate test-module basename and the initially inconsistent version were corrected without weakening behavior checks; failure logs are retained. New tests cover no-DP compilation, service mismatch, missing costs, exact break-even, whole-text token admission, TLS trust failures and deliberate data tampering.

A separate auditor imports no producer optimizer, selector or codec. It reconstructs full-initialization and wave costs for all 27 workloads, checks 36 serialized replays against original evidence digests and verdicts, reconstructs all codec byte sizes and diagnostic pages, and independently checks 27 planning scenarios and 131 summary comparisons. These checks validate recorded arithmetic and provenance, not third-party security certification. Release verification separately records extracted-source tests, the unchanged Future Code suite, installed-source hashes and archive integrity.

11 Deployment Recommendation, Limitations, and Next Experiments

11.1 Deployment recommendation

HACT v5.1 is an evidence-serialization component, and the recommendation is scoped accordingly: it applies where a service already requires protected verification evidence and hierarchical delivery,

not as a general agent or build accelerator. Choose a service before an algorithm. Do not deploy the strict wrapper solely on the strength of these transmission savings. For local trusted status, use the root projection; for final per-test states, a flat ledger may suffice. Where protected hierarchical evidence is already required, enable a suitable codec, retain the incumbent, and consider learned-order balanced fixed8 as the low-fitting-cost candidate. Exact DP remains an offline reference and can be worthwhile when its extra byte reduction pays back. Block DP is an ablation, not the default.

A cost calculator is not measured payback. The new scenarios expose when assumed extra costs dominate compressed delivery savings. They neither import a five-second overhead from a different fixture as though paired with each source case nor assume unlimited repeat exports. Dynamic adaptation is optional; the priced window selects already compiled candidates and can regress under rapid change. Whole-epoch compression changes streaming latency, and high-latency links do not necessarily favor HACT. Those are deployment questions, not consequences of the tree objective.

New default, new evidence boundary. The practical recommendation changed after the v4 review. It is therefore tested on new frozen synthetic seeds with disjoint train/validation/held-out draws. The twelve source candidates are reused diagnostic evidence and are not an independent test of model-generated repairs. On these replays, learned balanced trees are smaller than original-order fixed8 yet larger than exact learned layouts. The recommendation is about compilation simplicity, not universal superiority. Three seeds per setting support descriptive ranges, not precise generalization claims.

11.2 Limitations

Unclosed empirical requests: neither a physical WAN nor actual public-vocabulary BPE was evaluated in this release. Software/vocabulary retrieval failed, and no authorized pair of remote controlled hosts was available. The optional tools and local tests do not close these experimental gaps. No hosted model was called, and there is no new SWE-bench or natural historical-bug cohort. The actual SWE-bench harness uses isolated repository environments [20]; scripted source mutations are not equivalent. Source-level token counts, when measured later, will still not establish provider billing or better agent reasoning.

Closing the BPE gap. Three concrete steps would close the tokenization measurement gap without inflating its meaning. First, the supplied admission tool already implements exact content-only counting for `cl100k_base` and `o200k_base` via `tiktoken` [19]; it needs only a working vocabulary file to produce real counts. A reproducible packaging step—pinning the `tiktoken` wheel and its `.bpe` vocabulary to a hashed release—would make the measurement portable. Second, because the paper’s layout objective minimizes *bytes* not tokens, the BPE measurement should be reported as a diagnostic alongside the byte result, not as a replacement primary metric: a layout that reduces padded bytes may or may not reduce tokens under a specific BPE vocabulary, and both directions are informative. Third, the token count should be reported per-page with the framing budget r separated from payload, so that a reader can verify that the savings are not absorbed by framing overhead. None of these steps would establish a billing or reasoning improvement; they would only complete the admission tool’s stated purpose with a real vocabulary.

Trust and scope: Future Code remains a single-host supervisor. The guard and TLS utility are not a distributed consensus protocol or fault-tolerant deployment. A passing declared suite cannot establish absence of arbitrary defects. Authorization requires an uncompromised checker, correctly scoped inputs and protected reference state. A separate integration service must bind the verified candidate and integration head atomically; no PDF table or root JSON card substitutes for that step.

Changing the tree is not free. Our metric is steady-state refresh traffic. Constructing a new layout requires planning and initializing its internal records; switching may also create scheduling or audit overhead. If a proposed tree is expected to save $d > 0$ bytes per future episode and its migration costs B_{switch} bytes, the simplest break-even count is B_{switch}/d . This arithmetic is not a validated online switching policy. A deployable policy should estimate savings on independent recent batches, account for drift and planning cost in compatible units, and require a margin before replacement.

A small verification context is not a small total intelligence requirement. A conjunction of trusted gate verdicts has an exact compact aggregate. Arbitrary design tradeoffs, theorem discovery, research assessment, and natural-language correctness generally do not. An encoder supporting exact recovery of every bit of an unrestricted n -bit state without retrieval must distinguish 2^n states and needs at least n bits. Durable evidence and selective reads relocate information; they do not make it disappear. HACT therefore makes no claim that every difficult project can be solved by short independent conversations.

11.3 Toward a productivity claim: design and evaluation protocol

The review (HACT_v5_Review_and_Productivity_Redesign.pdf) recommends treating HACT as the evidence substrate of a substantially different program about *verified-frontier context retirement*, dependency-aware parallelism, and task granularity. We record that program here as a protocol, not as a completed experiment, and we give the concrete acceptance gates it must pass.

The central hypothesis is that validated task boundaries make selective forgetting and reuse safer and more economical than unconditional replay or length-only truncation. A task-local capsule carries typed fields: task and requirement IDs; candidate/head IDs and read-set bindings; write scope and interface versions; verified predicates and evidence references; unresolved questions and counterexamples; pending messages and ownership; and artifact references, budget, and next gate. Executable facts, source observations, and unverified model hypotheses must have different types: a test PASS does not certify the semantic completeness of a natural-language summary.

A defensible accounting identity (exact under its stated assumptions, not a guarantee) is that, with N model calls, fixed header h , and g retained tokens per call, complete replay submits $I_{\text{full}} = Nh + gN(N - 1)/2$, while renewing context every L calls with capsule size s , extra capsule construction tokens F , and renewal rework Q submits

$$I_{\text{renew}} = N(h + s) + gN(L - 1)/2 + F + Q. \quad (19)$$

The policy loses when F or Q is large. Against a good observation-masking baseline [16], the quadratic history term may already be absent, so this bookkeeping alone does not establish a productivity win.

Required causal ablations separate context policy from coordination policy and compare against the falsifying baselines: (i) a strong single agent with observation masking and focused retrieval; (ii) the same single agent with periodic summary/reset; (iii) the existing hierarchical Future Code bounded contexts; (iv) a decentralized verified-board baseline; (v) the proposed retirement with a fixed scheduler; and (vi) the scheduler without frontier-aware retirement. All main arms must use equal total budgets and the same final checks, sandbox and protected state; a many-agent system must not receive a multiplied budget unnoticed. The primary quality variable is whole-project acceptance under a frozen independent contract, reported as *cost per accepted project* next to acceptance rate, with deadline success and time-to-acceptance distributions retaining failures as censored observations.

A practical go/no-go gate, proposed (not measured) is at least a 20% reduction in cost at a prespecified quality margin, with the confidence interval, sample-size plan, timeout handling

and failed-run accounting fixed before outcomes. No benefit over masking, verified-board and fixed-scheduler baselines means the expensive new controller should not be promoted. If HACT’s tree adds no improvement after flattening evidence while keeping the contract, HACT remains optional and the productivity gain, if any, belongs to the context/scheduling layer.

Pre-registration and falsification closure. The protocol should be frozen in a versioned document before any live data collection: the six arms, the acceptance contract, the quality margin, the sample-size calculation, and the censoring rule for timed-out runs are all declared ex ante. Two explicit closure rules prevent indefinite iteration. *Null closure*: if the retirement arm’s cost-per-accepted-project is not at least 20% below the best baseline after the planned sample size, the productivity hypothesis is rejected and HACT is not promoted beyond its serialization role. *Attribution closure*: if the scheduler alone (arm vi) matches the retirement-plus-scheduler arm (arm v) within the pre-registered margin, the contribution is assigned to scheduling, not to frontier-aware retirement. Either closure terminates the protocol with a written result; no post-hoc arm addition or margin relaxation is permitted without a new pre-registered protocol.

Capsule integrity check. Each emitted capsule must carry a typed manifest listing verified predicates, unresolved questions, and artifact references; a post-episode auditor reconstructs the full task context from capsules alone and checks that the reconstructed context passes the same frozen acceptance contract. If the capsule reconstruction fails acceptance for any task in a completed project, that project is scored as a failure of the retirement policy, not as a serialization artifact. This ensures that information loss from context retirement is counted in the quality metric rather than hidden in a downstream re-check.

Until this live cohort is run, this direction is labeled a design and evaluation protocol; there is no justified replacement numerical abstract.

A Audit and Reproduction

A.1 Exact data generation

The synthetic generator uses NumPy’s `default_rng` initialized with a `SeedSequence` whose entries are `[260918, n, seed, stream]`. Stream zero produces training episodes and stream one held-out episodes. Group construction uses `SeedSequence([7001, seed, n])`. Pilot seeds 1000 and 1001 are excluded from all main tables. Scaling sizes other than 128 have their own saved records; the four reused 128-gate seeds are clearly identified in the analysis.

The v5 frozen contract fixes sizes 128, 512 and 1,024; families clustered, independent and global; and seeds 55011–55013, with 128 training, 128 validation and 256 held-out epochs per workload. The executable fixture uses `SeedSequence([83012, seed, stream])`. Interrupted fixture runs append completed snapshot records to a journal and replay those records when resuming; they do not reclassify incomplete work as a completed seed.

The main commands, from the extracted research root, are:

```
python -m pytest -q
python -m experiments.synthetic --family clustered --n 128 --seeds 20
python -m experiments.executable --seeds 6 --out results/executable_fresh
python -m experiments.sensitivity
python -m experiments.aggregate
python -m experiments.validate_results
```

Nothing in these commands contacts an LLM endpoint. The artifact separates raw results from regenerated tables.

A.2 Software and data identities

Archived runs used Python 3.13.5 on Linux x86_64; exact installed package and platform records accompany emitted reports. Compilers were measured with `OPENBLAS_NUM_THREADS=1`. No accelerator is required. Vendored upstream sources and tests are retained with their notices. Every fresh mutation is checked against its archived base-source hash, replacement span, and resulting source hash. This binds a result to a specific intervention, not a natural historical bug distribution.

B Certificate Encoding and Trust-Boundary Details

Each capsule contains `range`, `counts`, `binding`, `evidence`, and `registry`. Hash fields are 64-character lowercase SHA-256 hexadecimal encodings. Canonical JSON uses sorted keys, ASCII escapes, and compact separators. In a packet, the header occupies 512 bytes including its newline; each child card occupies 320 bytes including its newline. Padding makes byte accounting exact. Fixed padding is not asserted to be a token-optimal production encoding; a binary or tokenizer-aware encoding would define a new price function and require re-evaluation.

The trusted verifier API rejects mismatched bindings, invalid verdicts, and malformed trace hashes. It is intentionally not exposed as a tool through which an untrusted LLM can certify its own work. Returned display capsules are compared with canonical trusted records rather than accepted because their self-reported hashes appear valid. This local design demonstrates record integrity and freshness under its trust assumptions; distributed authenticated transport and credential isolation remain host-runtime responsibilities.

The optimizer’s $O(bn^3)$ claim concerns planning. The reference kernel prioritizes clarity and scans interval generation stamps; it also hashes tree descriptions. We do not infer an asymptotically optimal runtime-update implementation from the communication bound. Large-scale implementations could use range-maximum indexes and cached topology digests, but those optimizations are outside the evaluated prototype.

C Optional Fixed-Share Analysis: Not the Deployment Recommendation

This appendix retains the former tracking theorem and its assumptions. It is an optional comparator, not the practical selector. On the measured streams it loses to the deterministic priced-window rule in three regimes. We have not searched for a favorable adversarial stream after seeing those results, and make no claim that a tracking bound implies an empirical advantage.

Let a declared bound R cover each round’s loss spread, $\max_i \ell_t(i) - \min_i \ell_t(i) \leq R$. Reject a measurement exceeding it instead of clipping the cost. Define $z_{t,i} = (\ell_t(i) - \min_j \ell_t(j))/R$. Starting at $p_1(i) = 1/K$, update

$$q_{t+1}(i) = \frac{p_t(i) \exp(-\eta z_{t,i})}{\sum_j p_t(j) \exp(-\eta z_{t,j})}, \quad (20)$$

$$p_{t+1}(i) = (1 - \alpha)q_{t+1}(i) + \alpha/K, \quad 0 < \eta \leq 1, \ 0 \leq \alpha < 1. \quad (21)$$

Independent draws from each new distribution can waste migrations. Instead, conditional on $A_t = i$, retain i with probability $\min\{1, p_{t+1}(i)/p_t(i)\}$. If it is not retained, sample from the normalized positive residual $(p_{t+1} - p_t)^+$. This maximal coupling has marginal p_{t+1} and switch probability $\text{TV}(p_t, p_{t+1})$. Established dependent-sampling methods motivate this choice [10]; we do not claim it as a new sampling algorithm.

Theorem 5 (Catalogue tracking with migration). *Assume a frozen catalogue, a fixed exogenous sequence of full-information loss vectors, spread at most R , and $M_{ij} \leq M$. For any comparator path $u_{1:T}$ with s changes and $0 < \alpha < 1$, the coupled policy satisfies*

$$\mathbb{E}[L_T] - \sum_t \ell_t(u_t) \leq R \left(\frac{\log K + s \log(K/\alpha) + (T-1-s) \log(1/(1-\alpha))}{\eta} + \frac{\eta T}{8} \right) + M(T-1)(\eta + \alpha). \quad (22)$$

For $\alpha = 0$ the stationary-comparator bound has numerator $\log K$ and applies only to $s = 0$.

Proof. The fixed-share update is exponential weighting over paths with an initial uniform prior, self-transition mass at least $1 - \alpha$, and transition mass at least α/K . The negative log prior of a path with s switches is bounded by the numerator. The bounded exponential-moment inequality gives expected normalized service regret at most that numerator divided by η , plus $\eta T/8$. Since losses are exogenous, the weights are independent of the sampled action path, and the coupling preserves each unconditional marginal. Exponential reweighting changes total variation by at most η ; uniform mixing adds at most α . Thus the expected switch count is at most $(T-1)(\eta + \alpha)$. Multiplication by M and addition of service regret proves the bound. $\square$ $\square$

The bound is not a per-run improvement guarantee. Choosing fixed $\eta = .5$ and $\alpha = .01$, as in our frozen experiment, need not make this bound sublinear with increasing T .

D Block-Backbone Compiler: Retained Ablation

The exact compiler’s cubic cost makes frequent full recompilation inappropriate. For a supplied display order, divide the leaves into consecutive blocks of at most s leaves, and fix a compact b -ary macro-tree on those blocks. Optimize each block’s microtree using its restricted completion waves and strict-epoch initialization cost. Replace each macro leaf by its optimized microtree. No evaluation waves enter this process.

Proposition 6 (Exactness in the block-backbone class). *With display order, block boundaries and macro-tree fixed, independent exact optimization of the blocks minimizes the training objective over every layout in that restricted class. The dynamic-programming work is $O(bns^2)$ with $O(bs^2)$ reusable DP workspace, excluding order learning and wave preparation.*

Proof. Every macro node covers a fixed union of complete blocks, so its activation weight and packet price are constant across the class. All variable costs lie inside disjoint microtrees. The objective is a constant plus the sum of their independent objectives. Conditional exactness therefore gives the minimizing member. There are at most $\lceil n/s \rceil$ subproblems, each of size at most s , yielding the stated DP bound. Reusing the subproblem arrays yields the workspace bound. $\square$ $\square$

For the reference implementation, M recorded waves and L total wave memberships add filtering and weight-preparation work of $O(\lceil n/s \rceil L + Mns)$, and storage for the input, output and local $M \times s$ arrays. This is not a linear-time claim for the entire pipeline. No bounded approximation ratio against the unrestricted exact tree is asserted; held-out cost can deteriorate. The historical experiment froze $s = 32$ before measurement. Its negative result—block DP lost to same-order balanced fixed8 in held-out bytes—motivates excluding this compiler from the practical path.

E Information Separations

These results concern the information available to a selector. They are not hardness results for all adaptive tree algorithms, and they do not imply that a complete hypergraph must be recovered: interval hit statistics are sufficient for a fixed order and objective.

E.1 Marginals do not determine a co-invalidation optimum

Consider four ordered leaves, binary trees, and a constant internal packet price $c > 0$. Let $\mathcal{D}_A$ choose $\{0, 1\}$ or $\{2, 3\}$ equiprobably and $\mathcal{D}_B$ choose $\{0, 3\}$ or $\{1, 2\}$ equiprobably. All leaf marginal activation probabilities equal $1/2$ in both distributions. The root and every three-leaf interval are always activated. The five tree shapes fall into the following classes:

Ordered tree	J_A/c	J_B/c
$((0, 1), (2, 3))$	2	3
$(0, ((1, 2), 3)), ((0, (1, 2)), 3)$	3	2.5
$(0, (1, (2, 3))), (((0, 1), 2), 3)$	2.5	3

Optimal costs are $2c$ and $2.5c$. A deterministic marginal-only selector receives identical input and must choose the same class, incurring at least $c/2$ regret on one distribution. The last class is weakly dominated by the balanced tree. If a randomized selector chooses the balanced tree with probability q , its regrets are at least $(1 - q)c$ and $qc/2$. Their maximum is minimized at $q = 2/3$ and equals $c/3$, which is attainable for this instance. A fixed initialization price adds the same $3c$ to all binary shapes and does not alter the separation.

E.2 Joint invalidation does not determine progressive layout

Every epoch now invalidates all four checks, and execution order is $(0, 1, 2, 3)$ with singleton completion waves. In workload A , check 0 fails, so only its wave is executed. In workload B , check 3 fails, so all four checks execute. Initial invalidation is identical, even jointly. Every binary tree has three internal nodes; its cost in A is $(3 + \text{depth}(0))c$ and in B is $(3 + \sum_i \text{depth}(i))c$. The root-0 comb minimizes A at $4c$ but costs $12c$ in B , whereas the balanced tree minimizes B at $11c$ but costs $5c$ in A . An invalidation-only randomized selector using a root-0 comb with probability p incurs regrets at least $(1 - p)c$ and pc , whose maximum is at least $c/2$. A deterministic selector incurs at least c . This proves a separation between stale-set information and schedule-induced completion information; it does not prohibit a selector that observes both.

F Cost, Statistical, and Online Proof Details

F.1 Full-refresh lower bound and global negative control

For an n -leaf tree with m internal nodes, the number of edges is $n + m - 1$, so $\sum_v d_v = n + m - 1$. Therefore

$$W(L) = S(n - 1) + (H + S)m \geq S(n - 1) + (H + S) \left\lceil \frac{n - 1}{b - 1} \right\rceil. \quad (23)$$

The inequality follows from $n - 1 = \sum_v (d_v - 1) \leq m(b - 1)$. At $n = 64$, $b = 8$, a complete eight-child construction attains the bound with $m = 9$ and $W = 27,648$ bytes. Thus all selected global-control layouts can tie an analytically minimal full refresh. Strict initialization followed by one global completion wave costs $2W$. This explains the exact negative control without assuming every balanced tree at every n attains the lower bound.

F.2 Uniform statistical control at a fixed order

For one activation wave per episode and a fixed registry order, let $M_n = n(n+1)/2$ and $\epsilon = \sqrt{\log(2M_n/\delta)/(2m)}$. The interval hit indicators are bounded independent variables across independent episodes, though checks within an episode may be arbitrarily dependent. Hoeffding plus a union bound gives simultaneous interval-frequency error at most ϵ . For every feasible tree, $|\hat{J}(T) - J(T)| \leq W(T)\epsilon$. Inserting the empirical minimizer gives regret at most $[W(\hat{T}) + W(T^*)]\epsilon \leq 2(n-1)C\epsilon$. This is the original HACT bound, retained for attribution. It is not automatically valid uniformly over arbitrary data-selected permutations; the separate held-out catalogue bound in Proposition 3 addresses that selection step.

For strict epochs with at most q completion waves, a loose range is $B = (q+1)(n-1)C$. Under the evaluated sequential eight-record flush rule with at most one final record per obligation, $q \leq \lceil n/8 \rceil$. A narrower observed range is not automatically a proved bound for an unobserved deployment distribution.

F.3 Maximal coupling and horizon dependence

For consecutive distributions p, p' , the probability mass that remains at index i under the coupling is $\min(p_i, p'_i)$. The remaining mass is distributed as $(p' - p)^+$, so the new marginal is exactly p' and $\Pr(A_{t+1} \neq A_t) = \text{TV}(p, p')$. For exponential reweighting by $a_i = \exp(-\eta z_i)$ with $z_i \in [0, 1]$, the normalizer is at most one, hence $q_i \geq e^{-\eta} p_i$. Thus $\sum_i \min(p_i, q_i) \geq e^{-\eta}$ and $\text{TV}(p, q) \leq 1 - e^{-\eta} \leq \eta$. Mixing q with the uniform distribution by weight α changes total variation by at most α . The triangle inequality gives the switch bound in Theorem 5.

For $K \geq 2$, $s = 0$, and $\alpha = 0$, write $B_T = RT/8 + M(T-1)$. The bound is $R \log K/\eta + \eta B_T$. When $\sqrt{R \log K/B_T} \leq 1$, choosing that learning rate gives at most $2\sqrt{R \log K B_T}$ expected regret, sublinear for fixed R, M, K . This is an established style of switching-cost bound, not a new rate. A single empirical switch with estimated future saving d and anticipated horizon h satisfies the familiar break-even inequality $hd > M_{ij}$. This is arithmetic, not a drift theorem: savings may reverse before h events.

G Evidence-to-Claim Register

Claim	Evidence	Scope boundary
Exact ordered optimum	Theorem 2; exhaustive enumeration tests	Fixed order and price model
Validation after order	Proposition 3; Hoeffding bound	Fixed catalogue, no shift
Publication soundness	Theorem 4; conditional proof	Trusted checker, protected state
Practical byte reduction	Table 2; frozen seeds	Clustered workloads, 3 seeds
Source replay reductions	Table 3; 36 replays	Diagnostic, not fresh checks
Root status identical	Equation (15); 593–594 bytes	Fixed tokenizer, layout-independent
Break-even accounting	Equations (17)–(18)	Declared C, E, R , codec-matched
BPE tokenization gap	c1100k_base/o200k_base remain UNKNOWN; admission tool supplied	Vocabulary retrieval failed; no billing claim
Context-retirement protocol	Pre-registered design; null and attribution closure rules	Not measured; six arms, frozen contract

References

- [1] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001. <https://doi.org/10.1109/32.962562>.
- [2] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. *Proceedings of ISSTA*, 2015.
- [3] You Wang, Michael Pradel, and Zhongxin Liu. Names Are All You Need: Effective and Safe Regression Test Selection for Python. arXiv:2605.25356v1, May 2026.
- [4] Umut Acar, Guy Blelloch, Matthias Blume, Robert Harper, and Kanat Tangwongsan. A Library for Self-Adjusting Computation. *Electronic Notes in Theoretical Computer Science*, 148:127–154, 2006.
- [5] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build Systems à la Carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP), Article 79, 2018.
- [6] Hirendu Vaishnav and Massoud Pedram. Alphabetic Trees: Enumeration and Optimization with Applications to VLSI Design. Author-hosted manuscript, Sections 4.1–4.4; publication date not established from the retrieved copy.
- [7] Ziheng Shangguan, Aviv Yaish, and Dahlia Malkhi. Authenticated Data Structures for Dynamic Workloads. arXiv:2608.25206v1, August 2026.
- [8] Jonathan E. Atkins, Erik G. Boman, and Bruce Hendrickson. A Spectral Algorithm for Seriation and the Consecutive Ones Problem. *SIAM Journal on Computing*, 28(1):297–310, 1998.
- [9] Mark Herbster and Manfred K. Warmuth. Tracking the Best Expert. *Machine Learning*, 32:151–178, 1998.
- [10] Sascha Geulen, Berthold Vöcking, and Melanie Winkler. Regret Minimization for Online Buffering Problems Using the Weighted Majority Algorithm. *Proceedings of COLT*, pages 132–143, 2010.
- [11] Nicolò Cesa-Bianchi, Ofer Dekel, and Ohad Shamir. Online Learning with Switching Costs and Other Adaptive Adversaries. *Advances in Neural Information Processing Systems*, 26, 2013.
- [12] Zexun Wang. Proof-Carrying Agent Actions: Model-Agnostic Runtime Governance for Heterogeneous Agent Systems. arXiv:2606.04104v1, June 2026.
- [13] Zihao Zheng, Baichuan Li, Junyi Yao, and Jiayu Long. Engineering Reliable Commit Gates for Agentic AI: Cost-Aware Verification Portfolios under Common-Mode Data Failures. arXiv:2609.10969v1, September 2026.
- [14] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*, 2024.
- [15] OpenTelemetry contributors. Batch Processor documentation, official collector implementation. <https://github.com/open-telemetry/opentelemetry-collector/tree/main/processor/batchprocessor>.
- [16] Lindenbauer et al. The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management. arXiv:2508.21433v3, October 2025.

- [17] Sun et al. Scaling Long-Horizon LLM Agent via Context-Folding. arXiv:2510.11967.
- [18] Yuzhen Mao and Azalia Mirhoseini. Decentralized Multi-Agent Systems with Shared Context. arXiv:2606.10662v1, June 2026.
- [19] OpenAI contributors. tiktoken: BPE tokenization implementation and encoding registry. <https://github.com/openai/tiktoken>.
- [20] SWE-bench contributors. The Harness: Docker-based patch evaluation reference. <https://www.swebench.com/SWE-bench/reference/harness/>.
- [21] B. Constantine, G. Forget, R. Geib, and R. Schrage. Framework for TCP Throughput Testing. RFC 6349, August 2011.
- [22] Wassily Hoeffding. Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.